ADS205 Decision, Computation and Language

Lecture 3 – Regular Expressions and Regular Pumping Lemma

Dr Chunchuan Lyu and Dr Xinhong Chen



Xi'an Jiaotong-Liverpool University
西交利物浦大学

Recap

- DFAs & NFAs
- Closure of Regular Languages (Concatenation and Star)

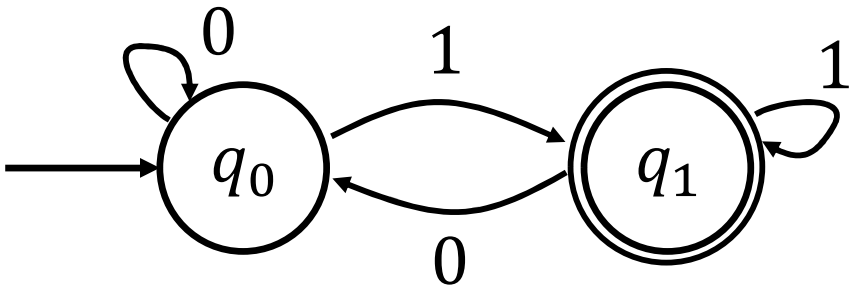
Today

- Regular Expression (Regex)
- Regexes ↔ FAs
- The Regular Pumping Lemma



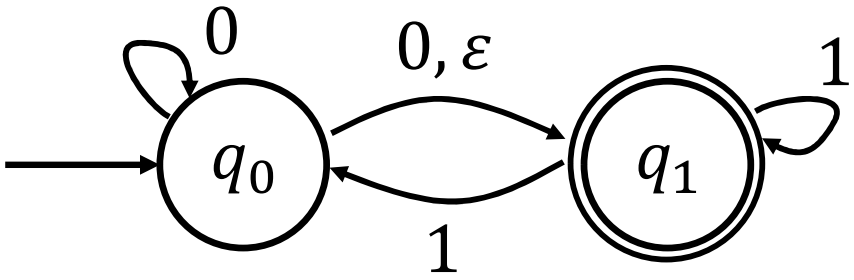
Recap: DFAs and NFAs

- In a Deterministic Finite Automata, **every** state has an outgoing transition for **all possible input characters**.



$\Sigma = \{0,1\}$

- In a Nondeterministic Finite Automata, there exist **any number** of **transitions** for each state.



$\Sigma = \{0,1\}$

- NFAs and DFAs are **<u>equivalent</u>**.

Recap: Regular Expression (Regex)

Regular Expression

An expression is <u>regular</u> if it is either atomic or composite of another regular expression

- Built from Σ, members $\Sigma, \emptyset, \varepsilon$ [Atomic]
- By using $\cup, \circ, *$ [Composite]

- Example: $A = \{w | w \; ends \; with \; 0\} \Rightarrow A = \Sigma^* 0$



Conversion between Regexes and FAs

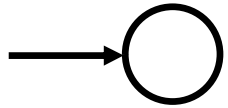
- For regular languages, it has multiple representation forms:
- Direct languages, such as $A = \{w | w\ ends\ with\ 0\}$
- Corresponding FAs
- Regexes

➢ **Regexes are shorter and easier for us to write when we describe languages**.

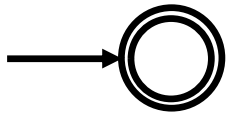


Regexes → FAs

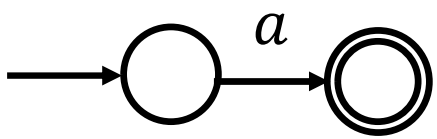
- Start by understanding Regexes
- Basic examples:
- $R = \emptyset$ => The language contains no string. => no string is accepted.



- $R = \varepsilon$ => The language has only the empty string.



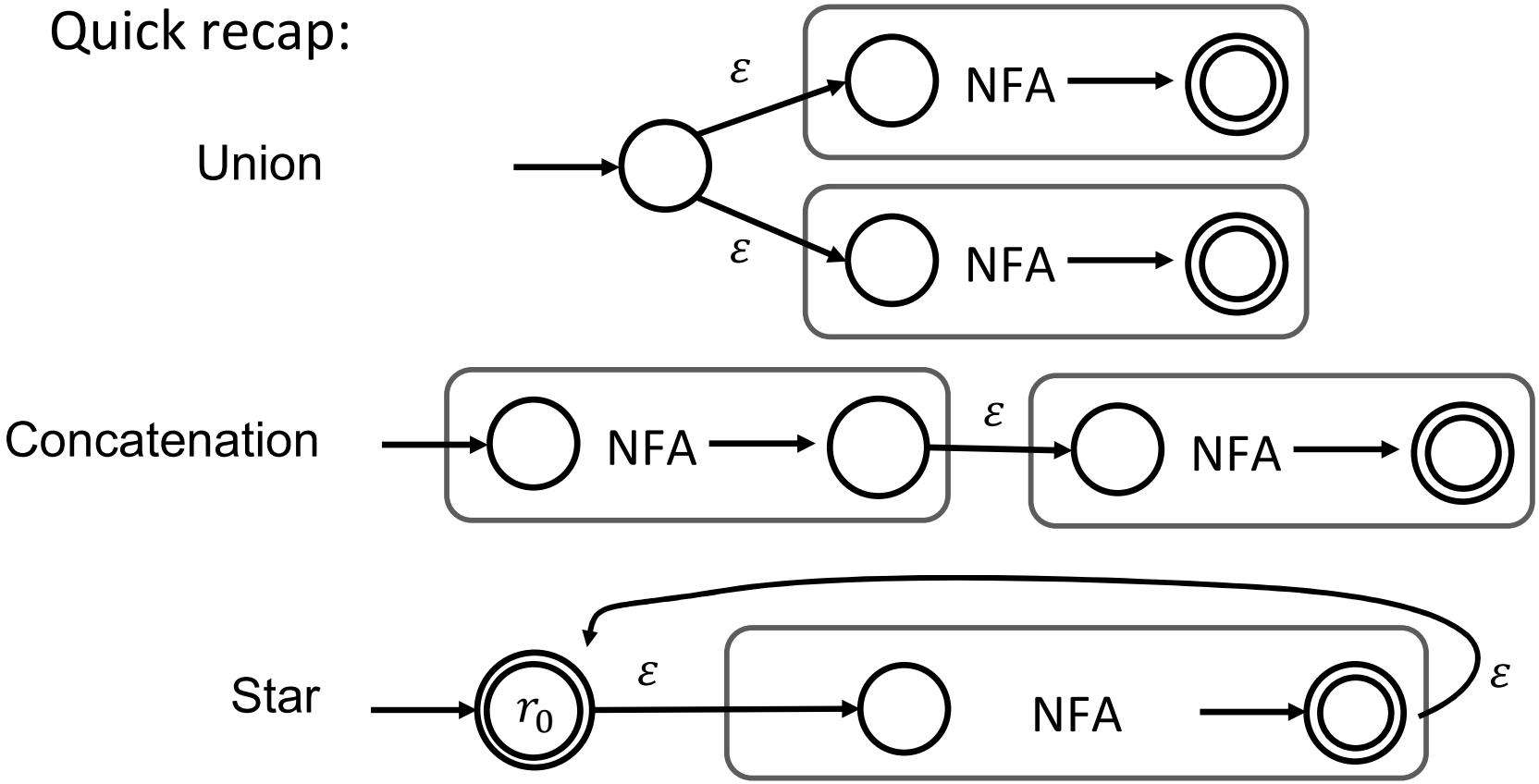
- $R = a$ => The language has only the string “a”.



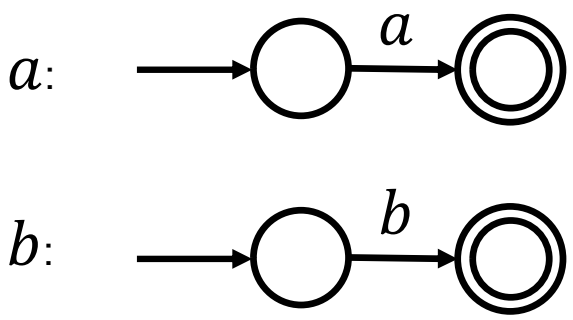


Regexes → FAs

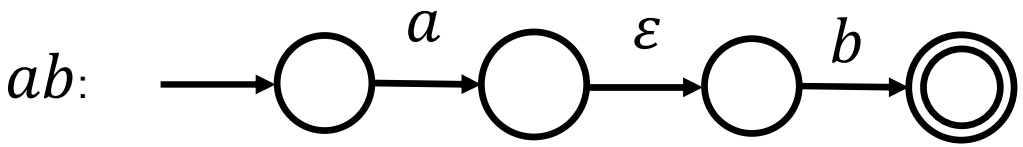
- **KEY: Step-by-step**
- Example: convert $(a \cup ab)^*$ into an equivalent NFA
- Quick recap:

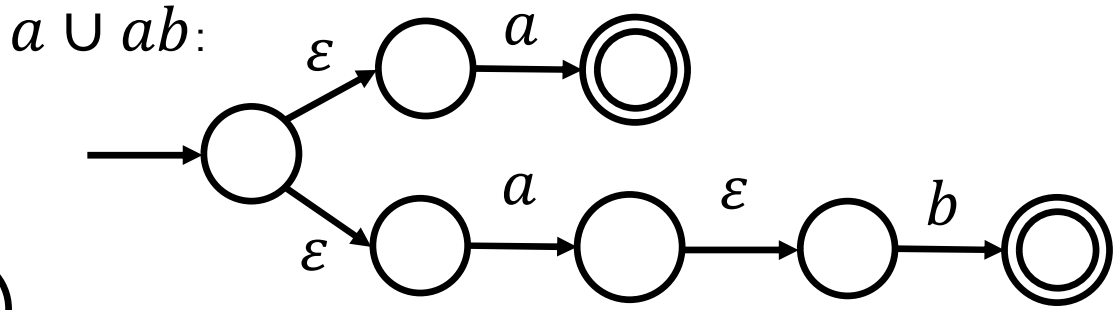


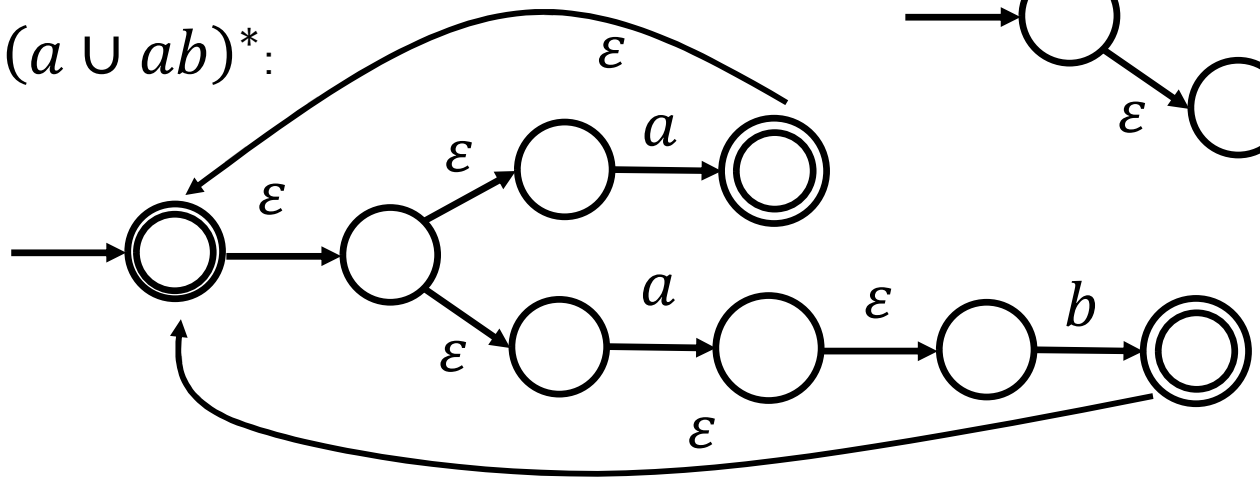
Regexes → FAs



- **KEY: Step-by-step**
- Example: convert $(a \cup ab)^*$ into an equivalent NFA
- (1) build basic component,
- (2) build composites







Regexes → FAs

- **KEY: Step-by-step**
- Example: convert $(a \cup ab)^*$ into an equivalent NFA
- (1) build basic component,
- (2) build composites

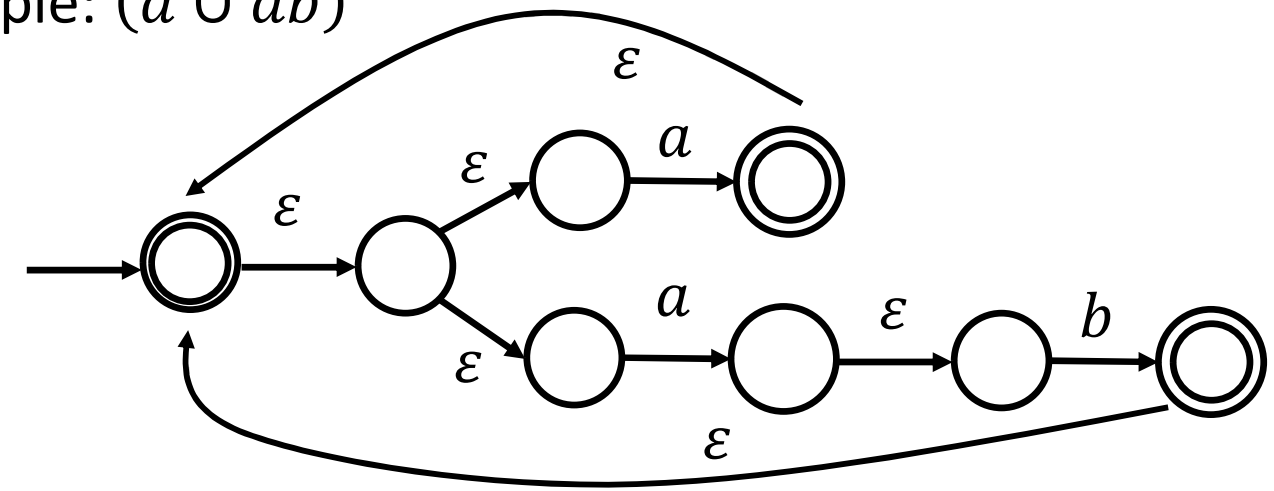
Recall: A language A is regular if some finite automaton recognizes it

Theorem: if R is a regular expression and $A = L(R)$, then A is regular.



FAs → Regexes

- FAs can grow large in an unbelievable speed.
- Example: $(a \cup ab)^*$

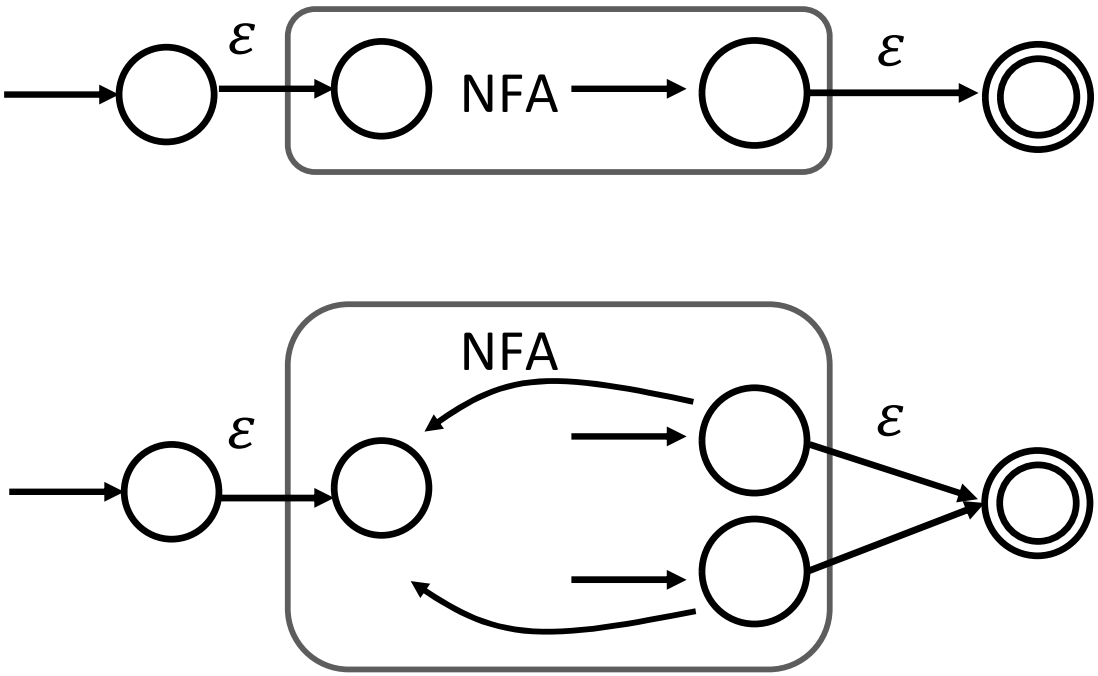


- Say now we have an input string: $(a \cup ab) \circ (a \cup ab)$
- If we could **judge based on the regexes** that this will be **accepted** by the FA, no need to go through the FAs!

➢ *Can we simplify a big NFA by replacing some parts to regexes?*

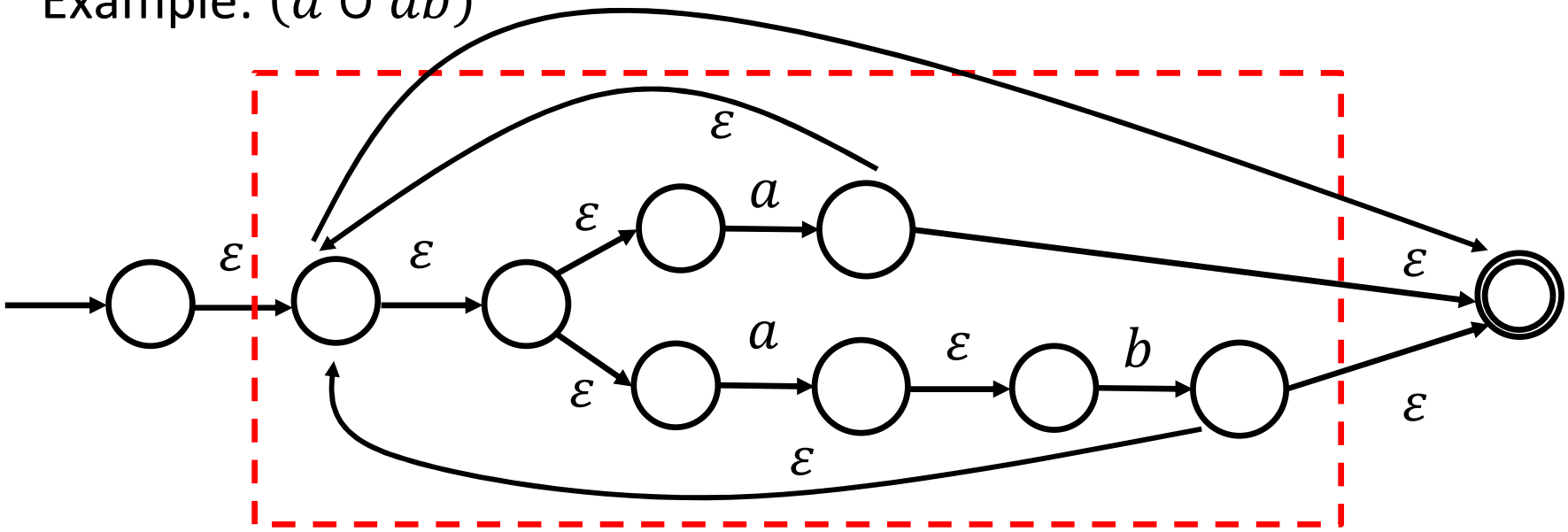
FAs → Regexes

- For each **NFA**, without loss of generality (wlog), there exists exactly **<u>one start state with no incoming transition</u>** and **<u>one accept state</u>**.

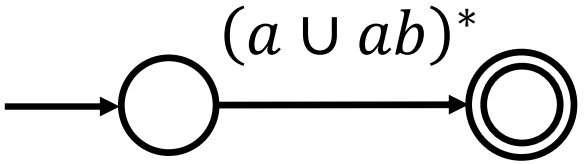


FAs → Regexes

- Example: $(a \cup ab)^*$



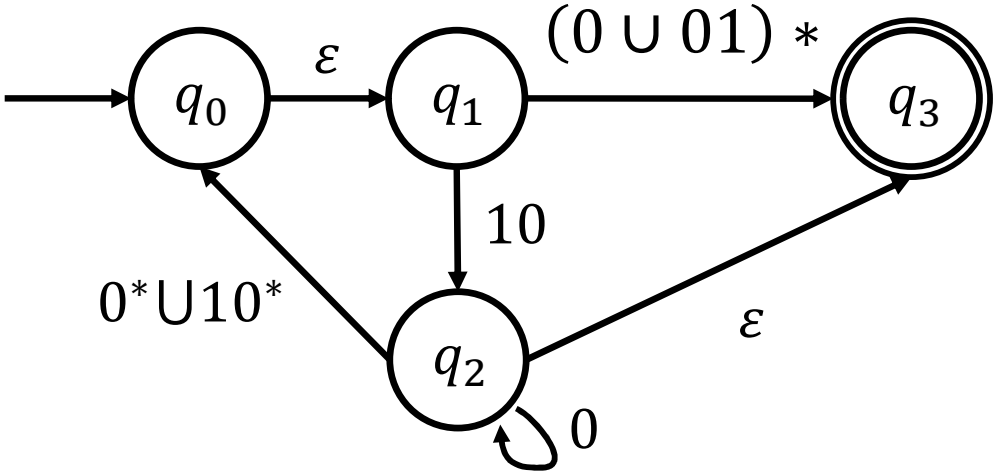
- Try to replace the middle part as the regex $(a \cup ab)^*$:
  - if the input string can be **accepted** by the FAs of the regex, then we **conduct the transition**



FAs → Regexes

- **A Generalized NFA** is similar to an NFA, but allows regular expressions as the transition labels.

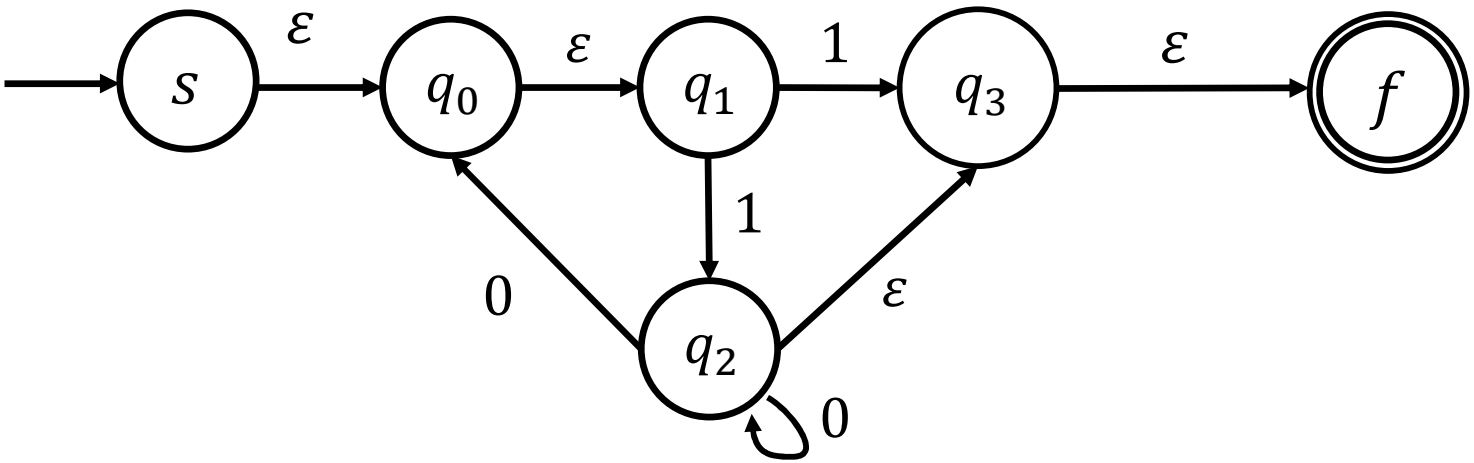
- Example:





FAs → Regexes

- **KEY: Still Step-by-Step**
- (1) add a start state and an accept state,
- (2) "Rip" a state, (3) add a corresponding regex-based transition,
- (4) repeat (2) and (3) until we have only the added states in step (1).

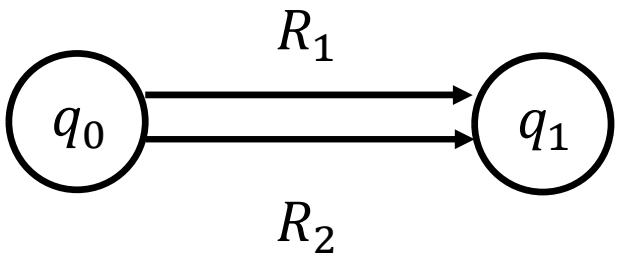


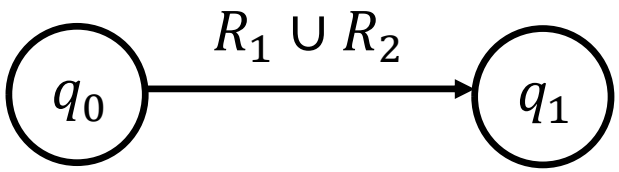


FAs → Regexes

- Basic "ripping" operations: recall regular operations ∪,∘,∗

① Multiple transitions between the same state pairs: if either one is accepted, the transition will be conducted => **union**

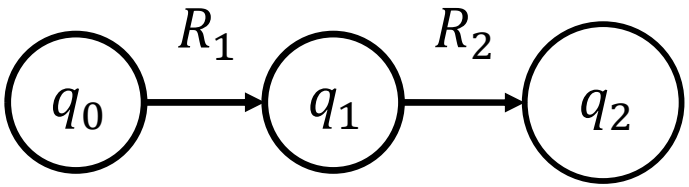


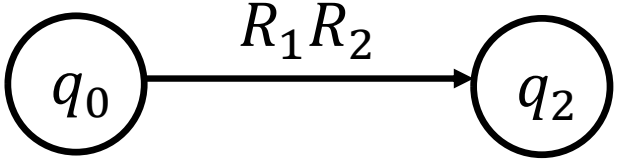


FAs → Regexes

- Basic “ripping” operations: recall regular operations ∪,∘,∗

② States are linked as a chain: if the regexes should be accepted sequentially => **concatenation**

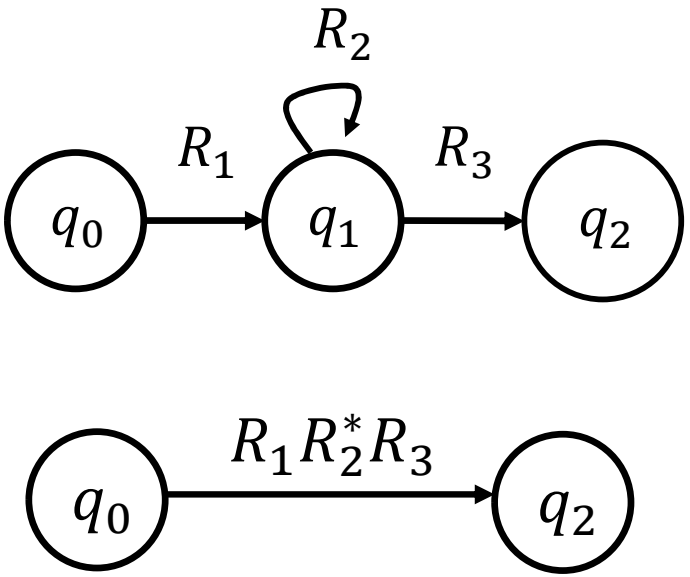




FAs → Regexes

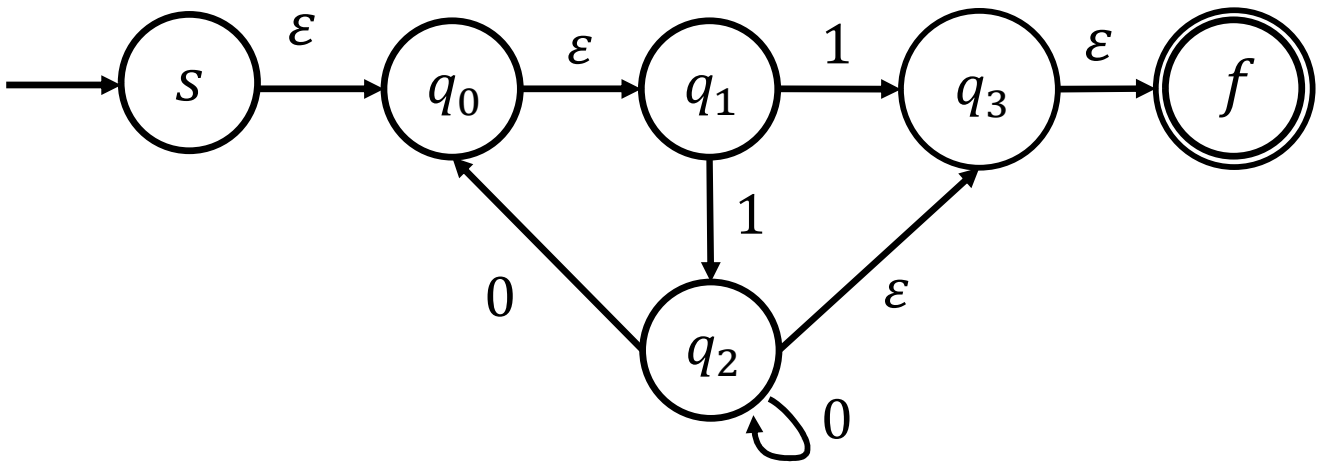
- Basic "ripping" operations: recall regular operations ∪,∘,∗

③ Self-loop: loop the regex itself multiple times => **star**



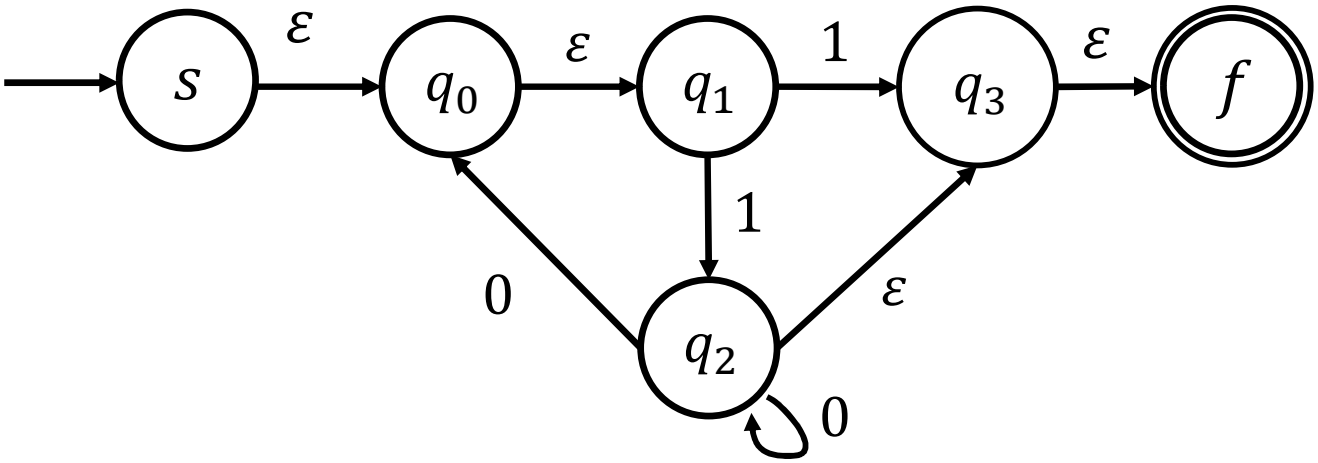
FAs → Regexes

- **KEY: Still Step-by-Step**
- (2) "Rip" a state, (3) add a corresponding regex-based transition,
- (4) repeat (2) and (3) until we have only the added states in step (1).



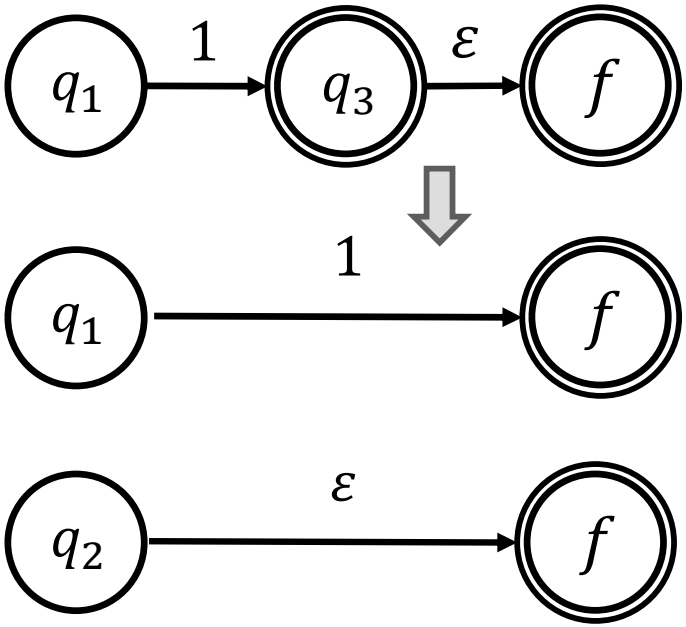
- "Rip" state q_3
- For each state directly linking to q_3, link it to the states q_3 links to

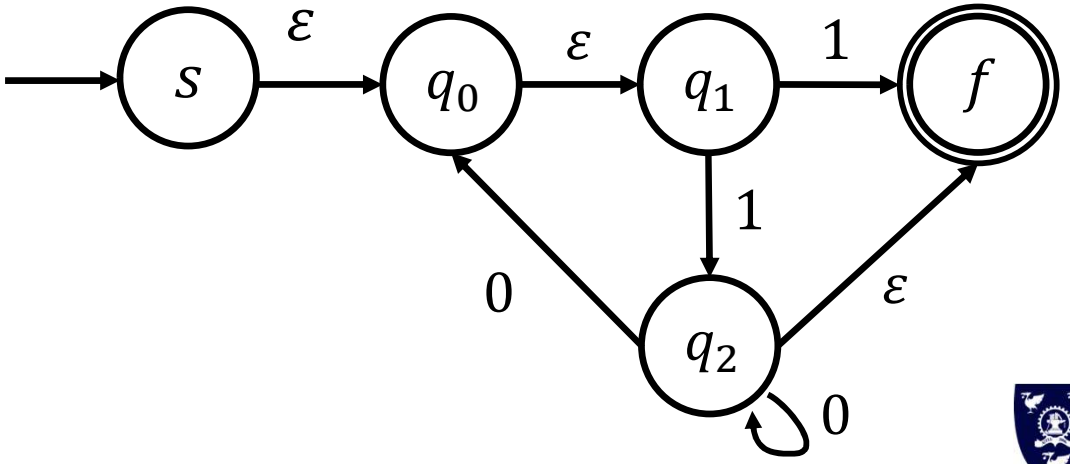
FAs → Regexes



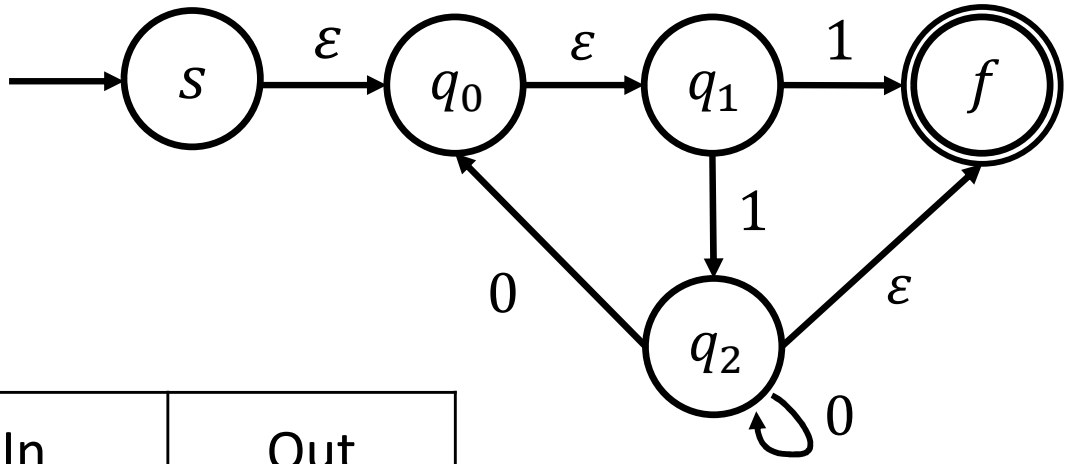
- "Rip" state q_3
- **In-and-out table** for q_3
- For each pair, add new transition

In	Out
q_1, q_2	f



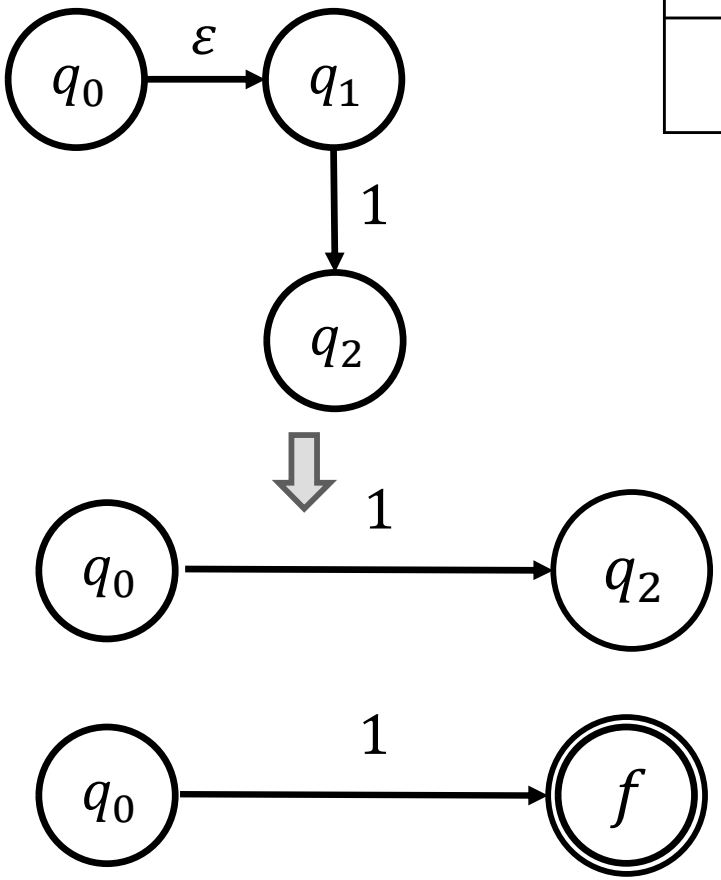


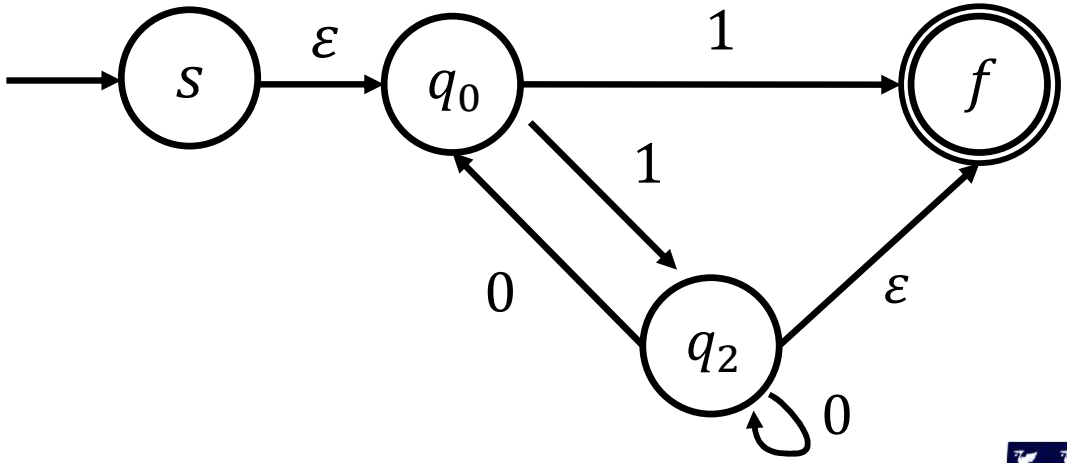
FAs → Regexes



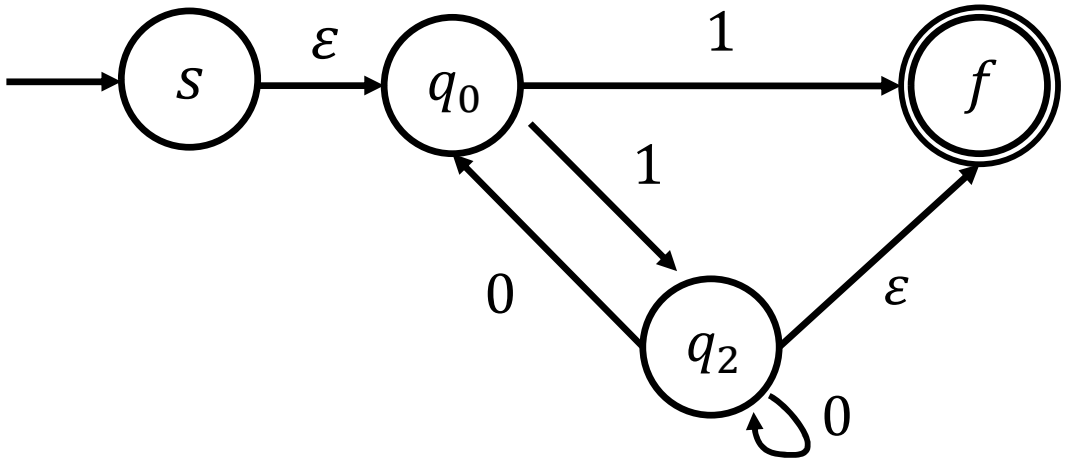
- "Rip" state q_1

In	Out
q_0	q_2, f

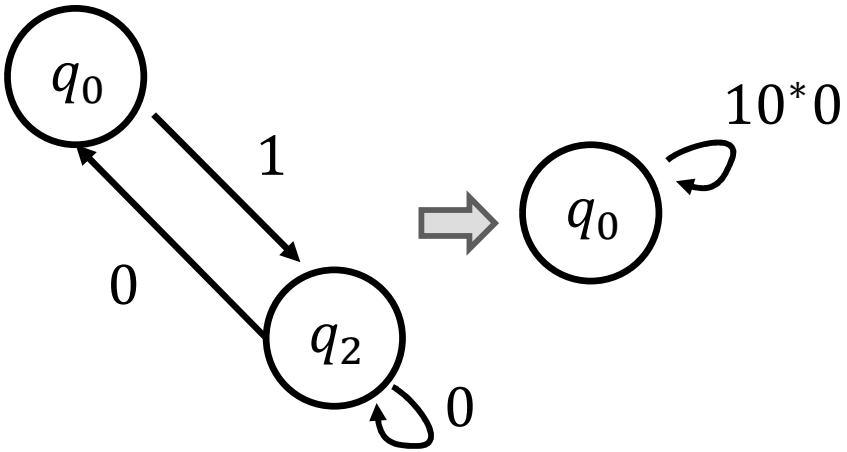




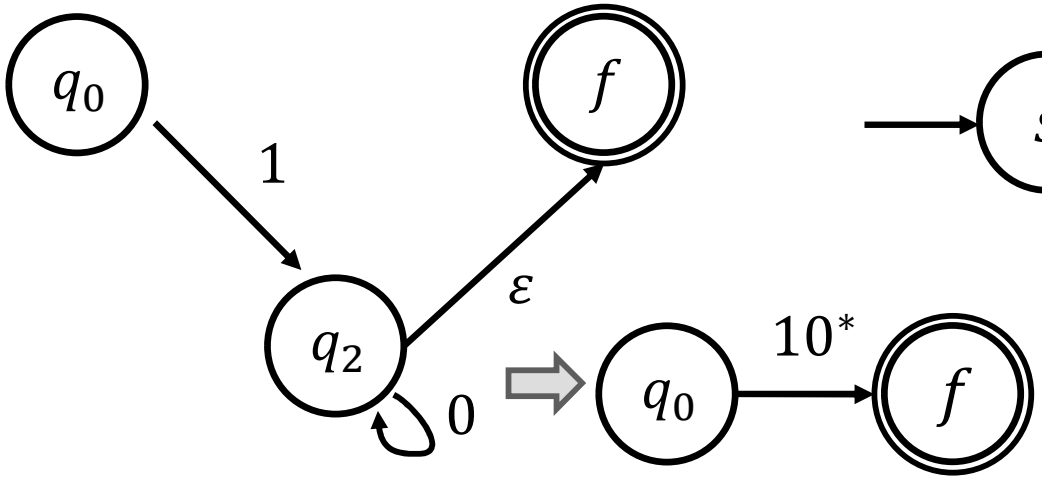
FAs → Regexes

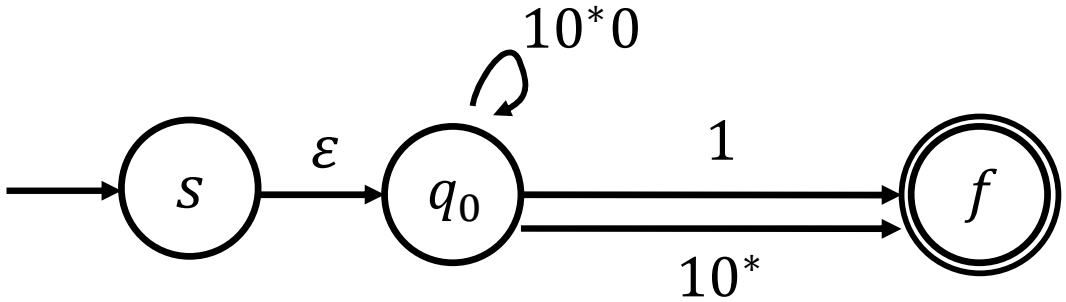


- "Rip" state q_2

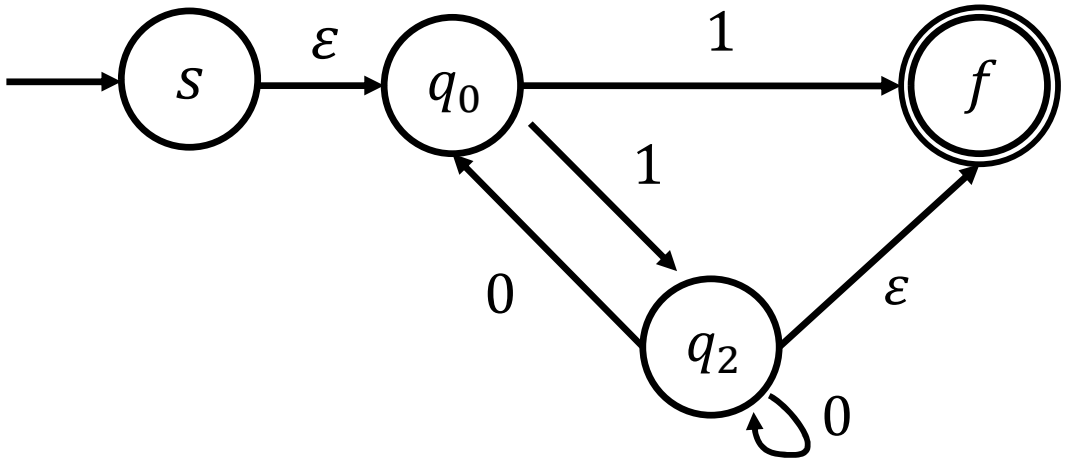


In	Out
q_0	q_0, f

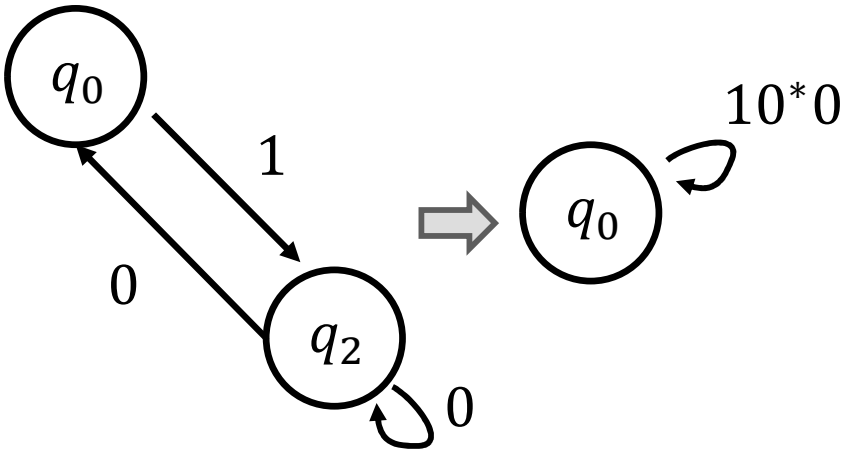




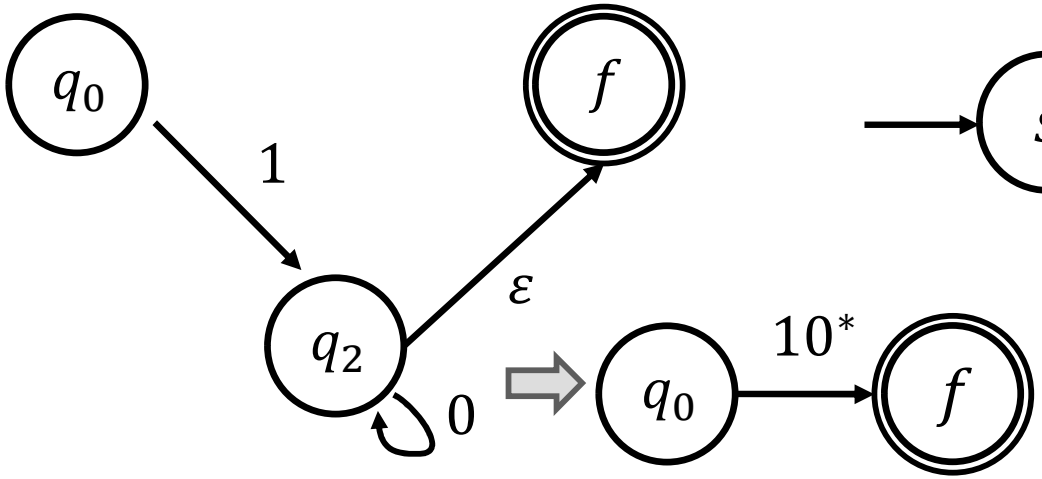
FAs → Regexes

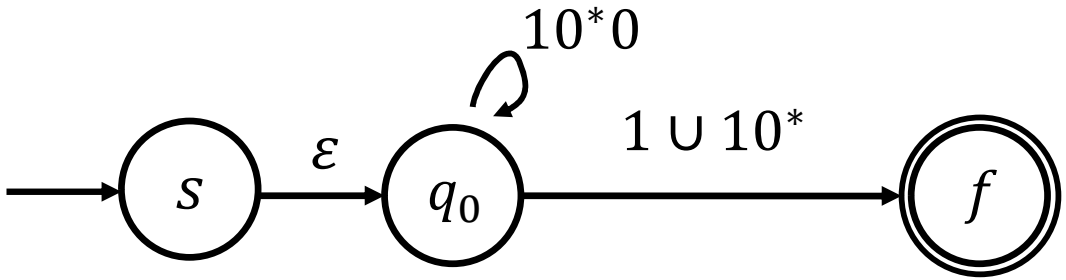


- "Rip" state q_2

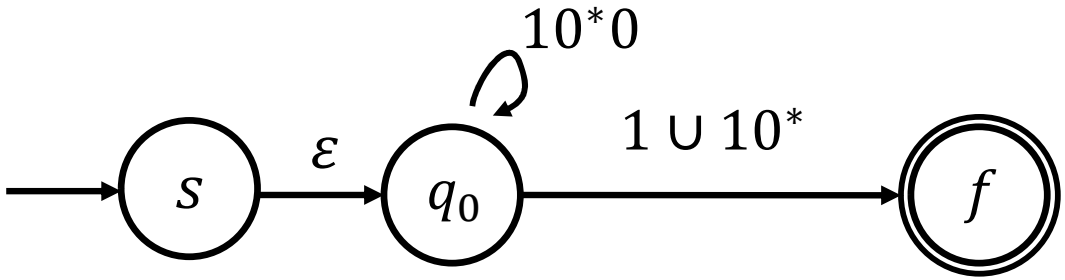


In	Out
q_0	q_0, f



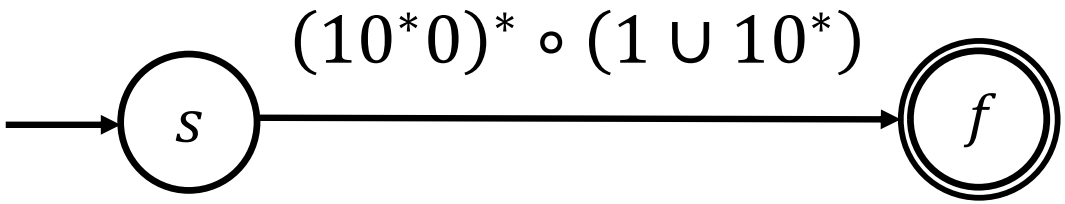


FAs → Regexes



- "Rip" state q_0

In	Out
s	f





FAs → Regexes

Recall: if R is a regular expression and $A = L(R)$, then A is regular.

Theorem: if a language A is regular, then $A = L(R)$ for some regular expression R.



Quick Review So Far

- FAs and Regular Language definition (all strings are accepted)
- Regular Operations and Regular Expressions
- DFAs, NFAs
- Equivalence between DFAs and NFAs
- Equivalence between Regular Expressions and FAs
- **Informally**, they are all equivalent, showing a regular language.
- Question: How to prove that a language is **NOT** regular?



How to prove that a language is NOT regular?

- To prove a language is regular, we draw a FA, write regex, etc.
- How to prove a language is not regular?
- Could not find a FA is not a good reason => how do we know we have tried every possibility?
- We need to find **some properties that only regular languages have**



How to prove that a language is NOT regular?

- Look back into DFAs, where each state should have an outgoing transition for all possible inputs
- Suppose a string w is **accepted** by a DFA M, and the length of w is **larger** than the number of states in M

➢ For DFA, M reads a character and makes a transition.

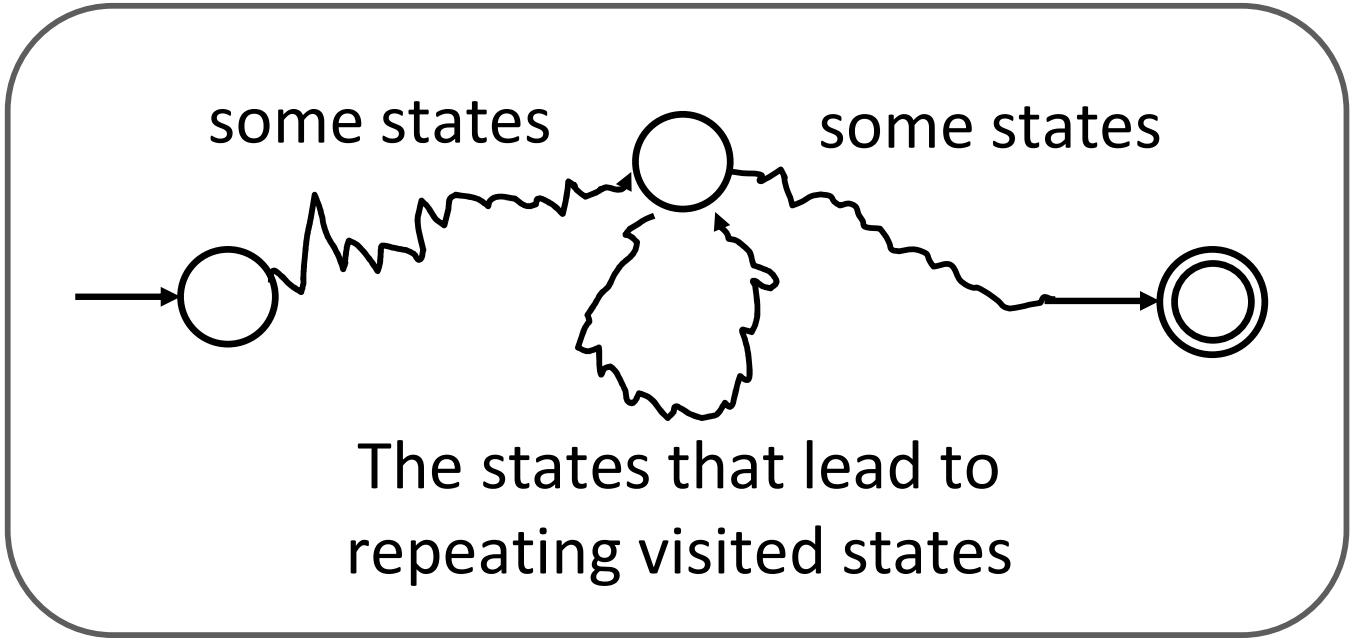
➢ Consider the most extreme case, every state is visited once, but w is still NOT done reading so not accepted yet.



How to prove that a language is NOT regular?

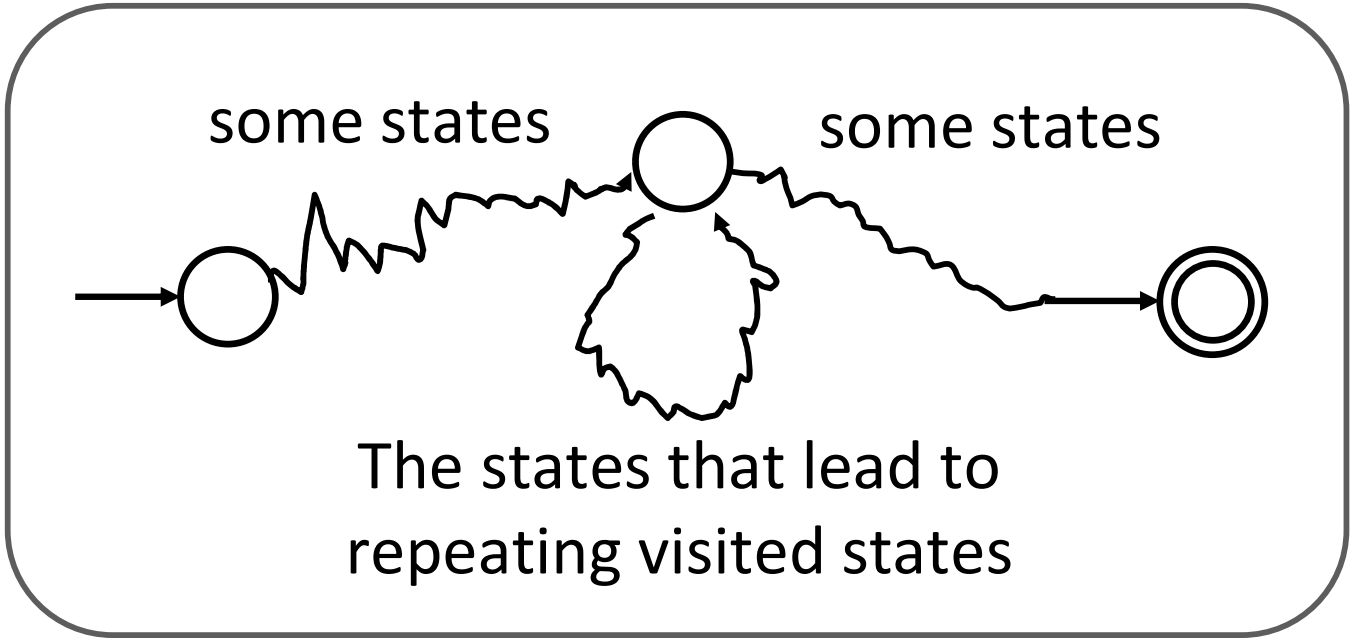
- Suppose a string w is **accepted** by a DFA M, and the length of w is **larger** than the number of states in M

➢ Some states are visited at least twice! => there is a loop!



How to prove that a language is NOT regular?

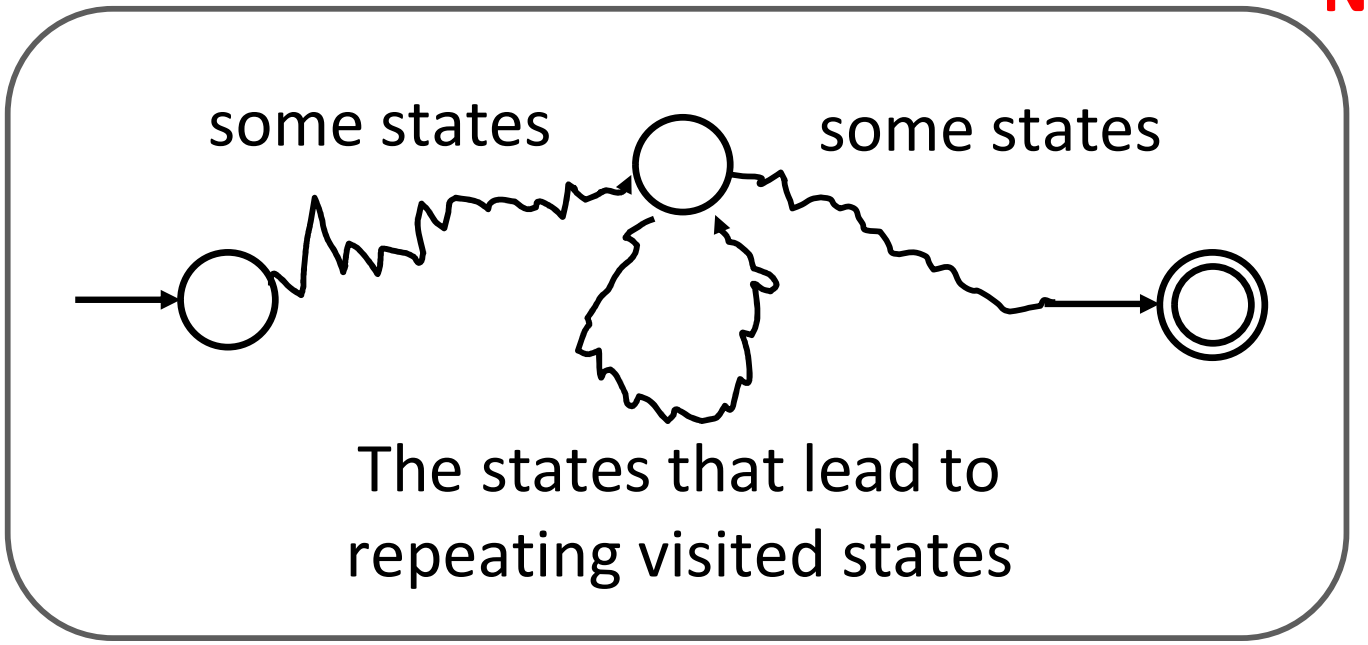
- Strings **looping for any times** are still **accepted** in M
- For regular languages, these strings created by looping should also still be in the languages.



How to prove that a language is NOT regular?

- Strings **looping for any times** are still **accepted** in M
- For regular languages, these strings created by looping should also still be in the languages. **=> What if they don't?**

Non-regular!



How to prove that a language is NOT regular?

① assume a language is regular, and find **a long enough string** that is in the language (”long-enough” is to allow the existence of loops)

② Create multiple strings by **modifying the looping part**

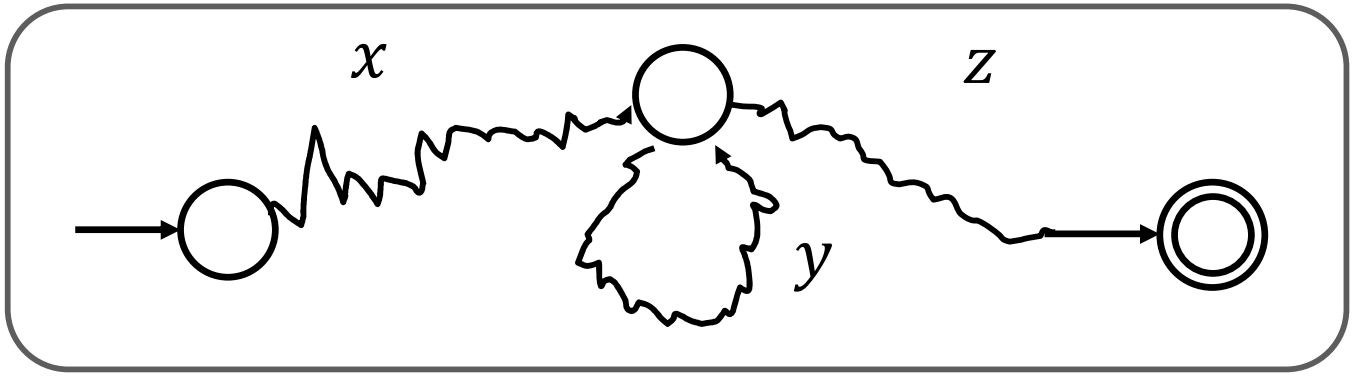
③ **Find a created string** that should not be in L.

④ Prove non-regularity.



How to prove that a language is NOT regular?

a regular language L



- a long-enough string $w = xyz, w \in L, y \neq \varepsilon$
- strings we could create by looping y: $\{w | w = xy^i z \ (i \geq 0)\}$
- we should have $xy^i z \in L$ for all $i \geq 0$
- How do we define "long-enough" mathematically?

➢ <u>As long as the length is longer than the # of states in DFA!</u>

The Regular Pumping Lemma

- For every regular language L, there is an integer p such that if $w \in L$ and $|w| \geq p$, then $w = xyz$ where:
- $xy^i z \in L$ for all $i \geq 0$
- $y \neq \varepsilon$
- $|xy| \leq p$



The Regular Pumping Lemma

- For every regular language L, there is an integer p …
- **why do we define p here?**
- The same language could correspond to multiple DFAs, so the number of states varies.
- The key is that if the language is regular, and **for all possible FAs**, the above analysis works consistently!



The Regular Pumping Lemma

- For every regular language L, there is an integer p such that if $w \in L$ and $|w| \geq p$, then $w = xyz$ where:
- $xy^i z \in L$ for all $i \geq 0$
- $y \neq \varepsilon$
- $\boldsymbol{|xy| \leq p}$
- the extreme case is $|xy| = p$ and $|z| = 0$.
- DFA has a start state, so a string with p characters will need $\boldsymbol{p + 1}$ **states** to process => "long-enough".



The Regular Pumping Lemma

- When proving non-regularity, **we don't need to know p** => the rule applies for all p
- Pay attention to **all possible breakups** of the input string $w = xyz$
- Only $|y| \geq 1$, **x and z could be empty.**
- for any i, if we could not make $xy^iz \in L$ **=> non-regular!**



The Regular Pumping Lemma

- Example:
- Prove $L = \{0^n 1^n : n \geq 0\}$ is not regular

① Assume L is regular, then there exists a p for L …, and all $w \in L$ with $|w| \geq p$, … (The pumping lemma)

② say we choose $w = 0^p 1^p = 000 \cdots 000111 \cdots 111$

③ Check **ALL possible breakups** of $w = xyz$, such as:

$$w = \underbrace{000 \cdots 000111 \cdots 111}_{x = 0^a \; y = 0^b, \quad z = 1^p}$$
$$b \geq 1$$



The Regular Pumping Lemma

$$w = \overbrace{000\cdots000}^{x = 0^a}\overbrace{111\cdots111}^{y = 0^b,\ z = 1^p}$$

$$b \geq 1, a + b = p$$

- Example:
- Prove $L = \{0^n 1^n : n \geq 0\}$ is not regular

④ Loop the repeated part to create strings $xy^i z, i \geq 0$

For example, $i = 2$, we have $xy^2z = 0^a 0^b 0^b 1^p = 0^{p+b} 1^p$

⑤ Check whether any string contradicts with L

If xy^2z is in L, then $p + b = p$, which is impossible

➢ L is not regular.



The Regular Pumping Lemma

- Example:
- Prove $L = \{0^{n^2} : n \geq 0\}$ is not regular

① Assume L is regular, then there exists a p for L …, and all $w \in L$ with $|w| \geq p$, … (The pumping lemma)

② say we choose $w = 0^{p^2}, |w| = p^2$

③ Check **ALL possible breakups** of $w = xyz$, such as $|x| = a, |y| = b, b \geq 1$ and z is the rest strings.



The Regular Pumping Lemma

- Example:
- Prove $L = \{0^{n^2} : n \geq 0\}$ is not regular

④ Pumping up the w, say $i = 2$, we have $|xyyz| = p^2 + b$, but $p^2 + b > p^2$, and $p^2 + b < (p+1)^2$ making $xyyz$ not in L

⑤ So L is not regular.



Review for today

- Regular Expressions and FAs are equivalent
- Proving a language is regular => write its regular expressions or construct its FAs
- **Don’t try some examples and then think we have proven regularity**
- Proving a language is not regular => use regular pumping lemma
- **Proving non-regularity only needs one example**



Q & A

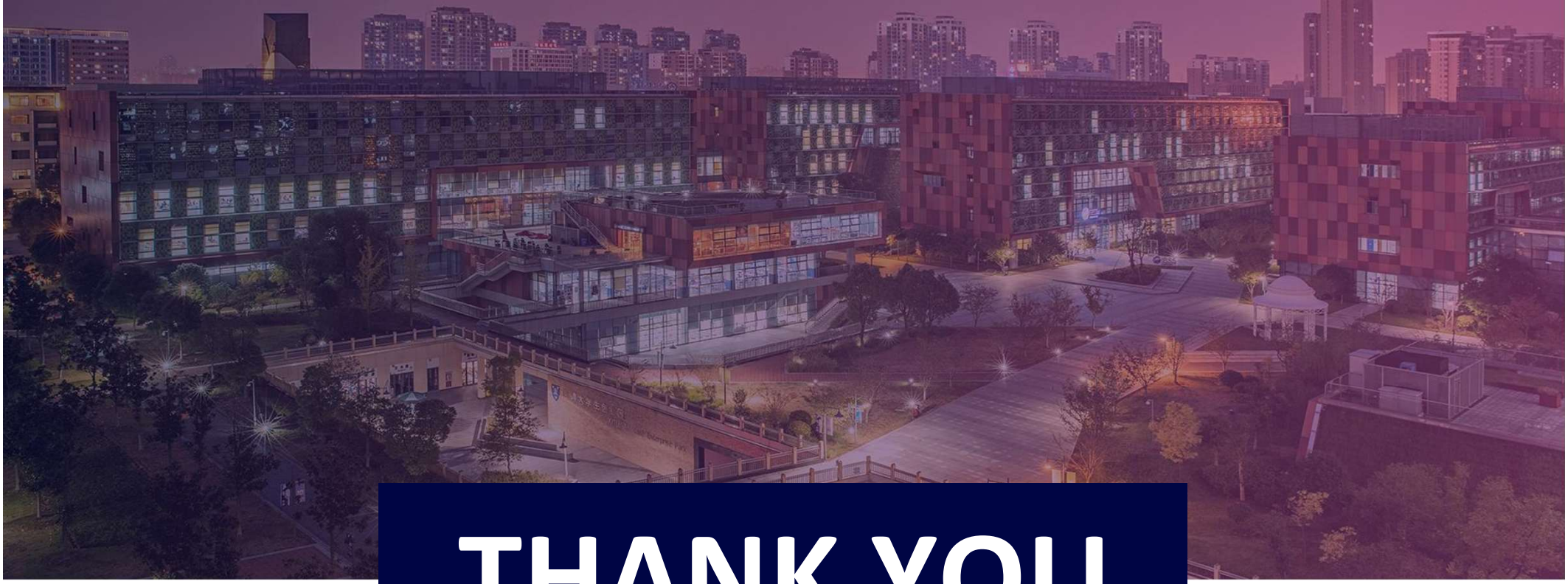
① If I cannot find a regular expression for a language, have I proved that it is not regular?

② Why do we only pump long strings?

③ If all the pumped versions of one string stay in L, is L regular?

④ Is $0^n 1^n$ a regular expression?





THANK YOU

Xi'an Jiaotong-Liverpool University
西交利物浦大学

XJTLU | SCHOOL OF FILM AND TV ARTS